OS Lab

Skill-20

To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

#define MAX_JOBS 20

typedef struct
{
   int job_id;
   pid_t pid;
   pid_t pgid;
   char command[100];
   char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;
int next_job_id = 1;

pid_t shell_pgid;
int shell_terminal;

/* Find a job using its job ID */
int find_job(int job_id)
{
   for (int i = 0; i < job_count; i++)
   {
      if (jobs[i].job_id == job_id)
      {
```

```
            return i;
        }
    }

    return -1;
}

/* Add stopped process to job table */
void add_job(pid_t pid, pid_t pgid, char *command)
{
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full\n");
        return;
    }

    jobs[job_count].job_id = next_job_id++;
    jobs[job_count].pid = pid;
    jobs[job_count].pgid = pgid;

    strcpy(
        jobs[job_count].command,
        command
    );

    strcpy(
        jobs[job_count].state,
        "Stopped"
    );

    job_count++;
}

/* Remove completed job */
void remove_job(int index)
{
    for (int i = index; i < job_count - 1; i++)
    {
        jobs[i] = jobs[i + 1];
    }

    job_count--;
}
```

```
/* Display job table */
void list_jobs()
{
   if (job_count == 0)
   {
      printf("No stopped jobs\n");
      return;
   }

   printf("\n");
   printf("Job ID\tPID\tPGID\tState\tCommand\n");
   printf("-----------------------------------------------\n");

   for (int i = 0; i < job_count; i++)
   {
      printf(
         "[%d]\t%d\t%d\t%s\t%s\n",
         jobs[i].job_id,
         jobs[i].pid,
         jobs[i].pgid,
         jobs[i].state,
         jobs[i].command
      );
   }

   printf("\n");
}

/* Resume a stopped job */
void resume_job(int job_id)
{
   int index = find_job(job_id);

   if (index == -1)
   {
      printf("No such job: %d\n", job_id);
      return;
   }

   Job *job = &jobs[index];

   printf(
      "Resuming job [%d]: %s\n",
      job->job_id,
```

```
   job->command
);

/*
 * Give terminal control to the job.
 */
if (tcsetpgrp(
       shell_terminal,
       job->pgid
   ) == -1)
{
   perror("tcsetpgrp");
}

/*
 * Send SIGCONT to the complete
 * process group.
 */
if (kill(
       -job->pgid,
       SIGCONT
   ) == -1)
{
   perror("SIGCONT");
}

strcpy(
   job->state,
   "Running"
);

int status;

/*
 * Wait until the process finishes
 * or becomes stopped again.
 */
waitpid(
   job->pid,
   &status,
   WUNTRACED
);

/*
```

```
     * Restore terminal control to shell.
     */
    if (tcsetpgrp(
            shell_terminal,
            shell_pgid
        ) == -1)
    {
        perror("tcsetpgrp");
    }

    if (WIFEXITED(status))
    {
        printf(
            "\n[%d] Completed\n",
            job->job_id
        );

        remove_job(index);
    }
    else if (WIFSIGNALED(status))
    {
        printf(
            "\n[%d] Terminated by signal %d\n",
            job->job_id,
            WTERMSIG(status)
        );

        remove_job(index);
    }
    else if (WIFSTOPPED(status))
    {
        strcpy(
            job->state,
            "Stopped"
        );

        printf(
            "\n[%d] Stopped again\n",
            job->job_id
        );
    }
}

/* Execute foreground command */
```

```
void execute_command(char *input)
{
   char command_copy[100];

   strcpy(
      command_copy,
      input
   );

   char *args[20];
   int argc = 0;

   char *token = strtok(
      input,
      " \t\n"
   );

   while (
      token != NULL &&
      argc < 19
   )
   {
      args[argc++] = token;

      token = strtok(
         NULL,
         " \t\n"
      );
   }

   args[argc] = NULL;

   if (argc == 0)
   {
      return;
   }

   pid_t pid = fork();

   if (pid < 0)
   {
      perror("fork");
      return;
   }
```

```
if (pid == 0)
{
    /*
     * Create a new process group.
     * The child becomes the group leader.
     */
    setpgid(
        0,
        0
    );

    /*
     * Child uses default terminal signals.
     */
    signal(
        SIGINT,
        SIG_DFL
    );

    signal(
        SIGTSTP,
        SIG_DFL
    );

    signal(
        SIGTTIN,
        SIG_DFL
    );

    signal(
        SIGTTOU,
        SIG_DFL
    );

    execvp(
        args[0],
        args
    );

    perror("execvp");
    exit(EXIT_FAILURE);
}
```

```
/*
 * Put child into its own process group.
 */
setpgid(
   pid,
   pid
);

/*
 * Give terminal control to child group.
 */
if (tcsetpgrp(
       shell_terminal,
       pid
   ) == -1)
{
   perror("tcsetpgrp");
}

printf(
   "Started PID=%d PGID=%d\n",
   pid,
   pid
);

int status;

/*
 * Wait for process to finish
 * or stop with Ctrl+Z.
 */
waitpid(
   pid,
   &status,
   WUNTRACED
);

/*
 * Restore terminal control to shell.
 */
if (tcsetpgrp(
       shell_terminal,
       shell_pgid
   ) == -1)
```

```
{
    perror("tcsetpgrp");
}

/*
 * Process was suspended by SIGTSTP.
 */
if (WIFSTOPPED(status))
{
    printf(
        "\nProcess stopped by signal %d\n",
        WSTOPSIG(status)
    );

    add_job(
        pid,
        pid,
        command_copy
    );

    printf(
        "[%d] Stopped    %s\n",
        jobs[job_count - 1].job_id,
        jobs[job_count - 1].command
    );
}

/*
 * Process completed normally.
 */
else if (WIFEXITED(status))
{
    printf(
        "\nProcess completed with status %d\n",
        WEXITSTATUS(status)
    );
}

/*
 * Process was terminated by a signal.
 */
else if (WIFSIGNALED(status))
{
    printf(
```

```
            "\nProcess terminated by signal %d\n",
            WTERMSIG(status)
        );
    }
}

/* Initialize shell process group */
void initialize_shell()
{
    shell_terminal = STDIN_FILENO;

    shell_pgid = getpid();

    /*
     * Put shell into its own process group.
     */
    setpgid(
        shell_pgid,
        shell_pgid
    );

    /*
     * Shell owns terminal initially.
     */
    tcsetpgrp(
        shell_terminal,
        shell_pgid
    );

    /*
     * Shell ignores terminal stop signals.
     */
    signal(
        SIGTSTP,
        SIG_IGN
    );

    signal(
        SIGTTIN,
        SIG_IGN
    );

    signal(
        SIGTTOU,
```

```
        SIG_IGN
    );
}

int main()
{
    char input[100];

    initialize_shell();

    printf("=== SIGTSTP and Process Group Demo ===\n");
    printf("\n");
    printf("Commands:\n");
    printf("  jobs       - display stopped jobs\n");
    printf("  resume N   - resume job N\n");
    printf("  exit       - exit shell\n");
    printf("\n");

    while (1)
    {
        printf("suspend$ ");
        fflush(stdout);

        if (fgets(
                input,
                sizeof(input),
                stdin
            ) == NULL)
        {
            break;
        }

        input[
            strcspn(input, "\n")
        ] = '\0';

        if (strlen(input) == 0)
        {
            continue;
        }

        if (strcmp(
                input,
                "exit"
```

```
            ) == 0)
        {
            break;
        }

        if (strcmp(
                input,
                "jobs"
            ) == 0)
        {
            list_jobs();
            continue;
        }

        if (strncmp(
                input,
                "resume ",
                7
            ) == 0)
        {
            int job_id = atoi(
                input + 7
            );

            resume_job(job_id);

            continue;
        }

        execute_command(input);
    }

    return 0;
}
```

```
lahari@DESKTOP-AIC33F7:~$ cd ~/OSLab
lahari@DESKTOP-AIC33F7:~/OSLab$ nano suspend_process.c
lahari@DESKTOP-AIC33F7:~/OSLab$ gcc suspend_process.c -o suspend_process
lahari@DESKTOP-AIC33F7:~/OSLab$ ./suspend_process
=== SIGTSTP and Process Group Demo ===

Commands:
  jobs       - display stopped jobs
  resume N   - resume job N
  exit       - exit shell

suspend$ sleep 30
Started PID=562 PGID=562
^Z
Process stopped by signal 20
[1] Stopped    sleep 30
suspend$ jobs

Job ID  PID     PGID    State   Command
------------------------------------------------
[1]     562     562     Stopped sleep 30

suspend$ resume 1
Resuming job [1]: sleep 30
```